



Das Kloster Maria Himmelfahrt zu Söflingen, ein Hort des Glaubens und der Stille, blickt auf eine ruhmreiche Vergangenheit zurück, die ihren Ursprung im Jahre des Herrn 1258 nahm. Einst als Zufluchtsort für die Klarissen gestiftet, hat dieser heilige Boden durch Jahrhunderte hindurch das Gebet und die christliche Nächstenliebe bewahrt. Trotz der Stürme der Zeiten, die über das Land zogen, blieben die uralten Mauern als ehrwürdige Zeugen einer tiefen Frömmigkeit bestehen, die bis auf den heutigen Tag in den Kreuzgängen und in der Stille der Gemäuer spürbar nachklingt.

Inmitten dieser segensreichen Stätte, dort wo einst das klösterliche Leben in Demut gedeihete, findet sich nunmehr die Meinloh-Grundschule. Es ist ein Ort, der in besonderer Weise vom Geiste der Vergangenheit durchdrungen ist: Die Schule schmiegt sich an den inneren Kern des Klosters, als würde sie von dem beständigen Segen und der geistigen Ausstrahlung der alten Mauern behütet. Die jungen Seelen, die hier täglich ein- und ausgehen, wandeln gleichsam unter dem sanften Schein einer spirituellen Aura, welche die Mühsal des Lernens mit der Ruhe der Ewigkeit verbindet und das Gebäude mit einer wundersamen, heilsamen Kraft durchdringt.

(c) George W. Aravidis - Ulm - Söflingen - Kloster Mariä Himmelfahrt

Επιμέλεια Θεμάτων Πανελλαδικών Εξετάσεων ΑΕΠΠ έτους 2017
Author: George W. Aravidis

Επίλυση Θεμάτων Πανελλαδικών Εξετάσεων 2017

ΘΕΜΑ Α.Α1

1-Σ

2-Λ

3-Λ

4-Σ

5-Σ

1-Σ

Αυτό συμβαίνει λόγω των **Νόμων του De Morgan** στη λογική (και την άλγεβρα Boole), οι οποίοι ορίζουν πώς μετασχηματίζεται η άρνηση μιας σύζευξης (ΚΑΙ) ή μιας διάζευξης (Ή).

Σύμφωνα με τους νόμους αυτούς, όταν εφαρμόζουμε την άρνηση σε μια έκφραση που περιέχει «ΚΑΙ»:

1. Το «ΚΑΙ» γίνεται «Ή».
2. Η άρνηση εφαρμόζεται ξεχωριστά σε κάθε όρο.

Έτσι, για την έκφραση **ΟΧΙ(A ΚΑΙ Β)**, η ισοδύναμη μορφή είναι **(ΟΧΙ Α) Ή (ΟΧΙ Β)**.

2-Λ

Η πρόταση είναι Ψευδής.

Η χρήση του διερμηνευτή (interpreter) δεν οδηγεί σε ταχύτερη, αλλά σε βραδύτερη εκτέλεση του προγράμματος σε σύγκριση με τη χρήση μεταγλωττιστή (compiler). Ο διερμηνευτής μεταφράζει και εκτελεί τις εντολές του πηγαίου προγράμματος μία προς μία κατά τη στιγμή της εκτέλεσης. Αυτό σημαίνει ότι αν μια εντολή βρίσκεται μέσα σε έναν βρόχο (loop), ο διερμηνευτής αναγκάζεται να τη μεταφράζει επανειλημμένα κάθε φορά που εκτελείται, γεγονός που καθυστερεί τη διαδικασία. Αντίθετα, ο μεταγλωττιστής μεταφράζει ολόκληρο το πρόγραμμα μία φορά, παράγοντας ένα αυτοδύναμο εκτελέσιμο αρχείο σε κώδικα μηχανής, το οποίο επιτυγχάνει πολύ ταχύτερη εκτέλεση καθώς δεν απαιτεί περαιτέρω μεταφραστική διαδικασία κατά την εκτέλεσή του.

3-Λ

Η πρόταση αυτή είναι Ψευδής.

Στη δομή επανάληψης «ΓΙΑ ... ΑΠΟ ... ΜΕΧΡΙ ... ΜΕ ...» (καθώς και σε όλες τις δομές επανάληψης τύπου *for*), ο έλεγχος της συνθήκης (δηλαδή το αν η τιμή της μεταβλητής ελέγχου βρίσκεται εντός των ορίων) πραγματοποιείται πριν από την εκτέλεση των εντολών που περιέχονται στο βρόχο.

Αν οι αρχικές τιμές που έχουν οριστεί είναι τέτοιες ώστε η συνθήκη να μην ικανοποιείται εξαρχής (για παράδειγμα, αν η τιμή έναρξης είναι μεγαλύτερη από την τιμή τερματισμού ενώ το βήμα είναι θετικό), τότε οι εντολές στο εσωτερικό της δομής δεν θα εκτελεστούν ούτε μία φορά.

Σε αντίθεση με τη δομή «ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ...ΜΕΧΡΙΣ_ΟΤΟΥ», η οποία εγγυάται τουλάχιστον μία εκτέλεση, η δομή «ΓΙΑ» είναι ελεγχόμενη από την αρχή, καθιστώντας την κατάλληλη για περιπτώσεις όπου ο αριθμός των επαναλήψεων είναι προκαθορισμένος και γνωστός πριν από την έναρξη του βρόχου.

4-Σ

Η πρόταση αυτή είναι Σωστή.

Η χρήση εμφωλευμένων (nested) εντολών ΑΝ συχνά οδηγεί σε «δυσανάγνωστο» κώδικα, ο οποίος περιέχει πολλά επίπεδα εσοχών (indentations), καθιστώντας δύσκολο τον εντοπισμό λογικών σφαλμάτων. Υπάρχουν δύο βασικοί τρόποι για να απλοποιήσουμε τέτοιες δομές:

1. **Χρήση της ΑΛΛΙΩΣ_ΑΝ:** Όταν έχουμε αλληλοαποκλειόμενες περιπτώσεις (π.χ. διαβάθμιση βαθμολογίας), η χρήση της ΑΛΛΙΩΣ_ΑΝ μετατρέπει την εμφωλευμένη δομή σε μια γραμμική δομή επιλογής. Αυτό κάνει τον κώδικα πιο καθαρό και εύκολο στην παρακολούθηση, καθώς κάθε περίπτωση εξετάζεται σε επίπεδο ίδιας ιεραρχίας.
2. **Χρήση Σύνθετων Εκφράσεων:** Πολλές φορές, πολλές εμφωλευμένες εντολές ΑΝ στην πραγματικότητα ελέγχουν συνδυασμούς συνθηκών που

μπορούν να ενωθούν με τους λογικούς τελεστές ΚΑΙ ή Ή. Αντί για «ΑΝ Α ΤΟΤΕ ΑΝ Β ΤΟΤΕ», μπορούμε να γράψουμε «ΑΝ Α ΚΑΙ Β ΤΟΤΕ».

Η απλοποίηση αυτή δεν βοηθά μόνο τον προγραμματιστή στη συντήρηση του κώδικα, αλλά καθιστά τη λογική του αλγορίθμου πιο άμεσα αντιληπτή, αποφεύγοντας την περιττή πολυπλοκότητα.

5-Σ

Η πρόταση αυτή είναι Σωστή.

Ο προγραμματισμός με υποπρογράμματα (διαδικασίες και συναρτήσεις) επιτρέπει τον τεμαχισμό ενός σύνθετου προβλήματος σε μικρότερα και διαχειρίσιμα τμήματα. Η αυτονομία αυτή προσφέρει σημαντικά οφέλη:

1. **Ανεξάρτητη Ανάπτυξη:** Κάθε υποπρόγραμμα μπορεί να γραφεί και να ελεγχθεί ξεχωριστά, χωρίς να είναι απαραίτητο να έχει ολοκληρωθεί ο κώδικας ολόκληρου του προγράμματος.
2. **Ευκολότερη Συντήρηση:** Όταν προκύπτει ανάγκη για διόρθωση ή αναβάθμιση, ο προγραμματιστής επικεντρώνεται μόνο στο συγκεκριμένο υποπρόγραμμα, χωρίς να επηρεάζεται η λειτουργία των υπόλοιπων τμημάτων του λογισμικού.
3. **Επαναχρησιμοποίηση:** Ένα υποπρόγραμμα που έχει αναπτυχθεί αυτόνομα μπορεί να χρησιμοποιηθεί σε διαφορετικά προγράμματα, μειώνοντας τον χρόνο ανάπτυξης και τον όγκο του κώδικα.

Η προσέγγιση αυτή ακολουθεί τη λογική «διαίρει και βασίλευε» (divide and conquer), η οποία αποτελεί τον ακρογωνιαίο λίθο της δομημένης ανάλυσης και επίλυσης προβλημάτων.

ΘΕΜΑ Α.Α2.α

Μια δομή δεδομένων είναι ένα σύνολο δεδομένων που συγκεντρώνονται και οργανώνονται σύμφωνα με ορισμένους κανόνες, ώστε να επιτρέπουν την εύκολη πρόσβαση, τροποποίηση και αναζήτηση των πληροφοριών. Δεν πρόκειται απλώς για την αποθήκευση τιμών, αλλά για τη λογική οργάνωση που καθορίζει πώς αλληλεπιδρούν τα δεδομένα μεταξύ τους.

Οι 4 βασικές λειτουργίες που μπορούν να εκτελεστούν επί των δομών δεδομένων είναι οι εξής:

1. **Δημιουργία (Creation):** Η αρχικοποίηση μιας δομής δεδομένων ώστε να είναι έτοιμη να δεχθεί στοιχεία.
2. **Προσπέλαση (Access/Traversal):** Η επίσκεψη σε κάθε στοιχείο της δομής, προκειμένου να επεξεργαστούμε ή να ανακτήσουμε τις τιμές του.
3. **Αναζήτηση (Searching):** Ο εντοπισμός ενός συγκεκριμένου στοιχείου μέσα στη δομή δεδομένων με βάση κάποιο κριτήριο (π.χ. αναζήτηση τιμής σε πίνακα).
4. **Ενημέρωση (Update):** Η τροποποίηση, η προσθήκη (εισαγωγή) ή η διαγραφή στοιχείων που περιέχονται στη δομή.

ΘΕΜΑ Α.Α2.β

Τα στοιχεία που προσδιορίζουν μια γλώσσα προγραμματισμού είναι τα εξής:

1. **Το Αλφάβητο:** Το σύνολο των χαρακτήρων που επιτρέπεται να χρησιμοποιηθούν (π.χ. γράμματα, αριθμοί, ειδικά σύμβολα).
2. **Το Λεξιλόγιο:** Το σύνολο των έγκυρων «λέξεων» ή συμβόλων που δημιουργούνται από το αλφάβητο (π.χ. δεσμευμένες λέξεις όπως ΑΡΧΗ, ΔΙΑΒΑΣΕ).
3. **Το Συντακτικό:** Το σύνολο των κανόνων που καθορίζουν πώς συνδυάζονται οι λέξεις για να σχηματίσουν έγκυρες εντολές ή προτάσεις.
4. **Η Σημασιολογία:** Το σύνολο των κανόνων που καθορίζουν το νόημα ή το αποτέλεσμα που παράγει η κάθε έγκυρη εντολή του προγράμματος.

ΘΕΜΑ Α.Α3

Επανάληψη	Συνθήκη ($i \leq k$)	Τιμή i	Τιμή k
1η	$0 \leq 12$ (Αληθής)	2	11
2η	$2 \leq 11$ (Αληθής)	4	10
3η	$4 \leq 10$ (Αληθής)	6	9
4η	$6 \leq 9$ (Αληθής)	8	8
5η	$8 \leq 8$ (Αληθής)	10	7
6η	$10 \leq 7$ (Ψευδής)	-	-
...			

Ανάλυση:

- Στην 1η επανάληψη: Το i γίνεται $0 + 2 = 2$ και το k γίνεται $12 - 1 = 11$. Εκτυπώνονται: 2, 11.
- Στην 2η επανάληψη: Το i γίνεται $2 + 2 = 4$ και το k γίνεται $11 - 1 = 10$. Εκτυπώνονται: 4, 10.
- Στην 3η επανάληψη: Το i γίνεται $4 + 2 = 6$ και το k γίνεται $10 - 1 = 9$. Εκτυπώνονται: 6, 9.
- Στην 4η επανάληψη: Το i γίνεται $6 + 2 = 8$ και το k γίνεται $9 - 1 = 8$. Εκτυπώνονται: 8, 8.
- Στην 5η επανάληψη: Το i γίνεται $8 + 2 = 10$ και το k γίνεται $8 - 1 = 7$. Εκτυπώνονται: 10, 7.
- Μετά την 5η επανάληψη: Ο έλεγχος $i \leq k$ ($10 \leq 7$) είναι ψευδής, οπότε ο βρόχος τερματίζεται.

ΘΕΜΑ Α.Α4.α

Στη δομή «ΟΣΟ», ο έλεγχος γίνεται πριν από την πρώτη εκτέλεση, ακριβώς όπως στη «ΓΙΑ».

```
S ← 0
i ← 5
ΟΣΟ i ≤ 20 ΕΠΑΝΑΛΑΒΕ
    ΔΙΑΒΑΣΕ X
    S ← S + X
    i ← i + 3
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

ΘΕΜΑ Α.Α4.β

```
S ← 0
i ← 5
ΑΝ i ≤ 20 ΤΟΤΕ
    ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
        ΔΙΑΒΑΣΕ X
        S ← S + X
        i ← i + 3
    ΜΕΧΡΙΣ_ΟΤΟΥ i > 20
ΤΕΛΟΣ_ΑΝ
```

Η προσθήκη της εντολής **ΑΝ** $i \leq 20$ τότε είναι απαραίτητη για να διασφαλίσουμε την απόλυτη ισοδυναμία με τη δομή «ΓΙΑ» σε περίπτωση που το αρχικό διάστημα δεν είναι έγκυρο.

ΘΕΜΑ Β.Β1

Για να εμφανίζονται οι τιμές 4, 8, 16, 20, 28, 32, 40, παρατηρούμε τα εξής:

- Οι αριθμοί είναι πολλαπλάσια του 4.
- Ξεκινάνε από το 4 και φτάνουν έως το 40.
- Λείπουν οι αριθμοί 12, 24, 36 (οι οποίοι είναι πολλαπλάσια του 4 αλλά και του 12).

Η λογική του αλγορίθμου είναι να διατρέξει όλους τους αριθμούς ανά 4 από το 4 έως το 40 και να εκτυπώσει μόνο εκείνους που δεν είναι πολλαπλάσια του 12.

```
i <- 4
ΟΣΟ i <= 40 ΕΠΑΝΑΛΑΒΕ
  ΑΝ i MOD 12 <> 0 ΤΟΤΕ
    ΓΡΑΨΕ i
  ΤΕΛΟΣ_ΑΝ
i <- i + 4
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
```

ΘΕΜΑ Β.Β2.α

Γραμμή 6 (Λογικό λάθος): Η μεταβλητή P (γινόμενο) αρχικοποιείται στο 0. Για να υπολογίσουμε γινόμενο, πρέπει να αρχικοποιείται στο 1, αλλιώς το αποτέλεσμα θα είναι πάντα 0.

Γραμμή 9 (Συντακτικό & Λογικό λάθος):

Συντακτικά: Η έκφραση $\text{MOD } 5 = 0$ είναι ημιτελής· έπρεπε να αναφέρεται ξανά στη μεταβλητή X ($X \text{ MOD } 5 = 0$).

Λογικά: Το πρόβλημα ζητά αριθμούς που είναι πολλαπλάσια του 3 και του 5 συγχρόνως, άρα ο σύνδεσμος πρέπει να είναι το ΚΑΙ και όχι το «Η».

Γραμμή 11 (Συντακτικό λάθος): Υπάρχει επιπλέον εντολή Τέλος_επανάληψης.
Η δομή «Για» κλείνει μόνο μία φορά στη γραμμή 12.

ΘΕΜΑ Β.Β2.β

ΠΡΟΓΡΑΜΜΑ Αριθμοί

ΜΕΤΑΒΛΗΤΕΣ

ΑΚΕΡΑΙΕΣ : X, P, i

ΑΡΧΗ

P ← 1

ΓΙΑ i **ΑΠΟ** 1 **ΜΕΧΡΙ** 10

ΔΙΑΒΑΣΕ X

ΑΝ X **MOD** 3 = 0 **ΚΑΙ** X **MOD** 5 = 0 **ΤΟΤΕ**

P ← P * X

ΤΕΛΟΣ_ΑΝ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΓΡΑΨΕ P

ΤΕΛΟΣ_ΠΡΟΓΡΑΜΜΑΤΟΣ

ΘΕΜΑ Γ

ΠΡΟΓΡΑΜΜΑ Θέμα_Γ_Πρωτάθλημα_Βόλει

! Γ1.α: Κατάλληλο τμήμα δηλώσεων

ΜΕΤΑΒΛΗΤΕΣ

ΑΚΕΡΑΙΕΣ : i, j, k, A[5,3], ΣΧ1, ΣΧ2, ΣΕΤ1, ΣΕΤ2, temp_A

ΧΑΡΑΚΤΗΡΕΣ : ON[5], temp_ON

ΑΡΧΗ

! Γ1.β: Διάβασμα ονομάτων σχολείων

ΓΙΑ i **ΑΠΟ** 1 **ΜΕΧΡΙ** 5

ΔΙΑΒΑΣΕ ON[i]

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

! Γ1.γ: Αρχικοποίηση του πίνακα A

ΓΙΑ i **ΑΠΟ** 1 **ΜΕΧΡΙ** 5

ΓΙΑ j **ΑΠΟ** 1 **ΜΕΧΡΙ** 3

A[i, j] <- 0

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

! Γ2: Διάβασμα δεδομένων 10 αγώνων και ενημέρωση

ΓΙΑ k **ΑΠΟ** 1 **ΜΕΧΡΙ** 10

! Διαβάζουμε τα δύο σχολεία και τα σετ που πήρε το καθένα

ΔΙΑΒΑΣΕ ΣΧ1, ΣΧ2, ΣΕΤ1, ΣΕΤ2

! Ενημέρωση των σετ υπέρ (στήλη 2)

! και κατά (στήλη 3) και για τα δύο σχολεία

A[ΣΧ1, 2] <- A[ΣΧ1, 2] + ΣΕΤ1

A[ΣΧ1, 3] <- A[ΣΧ1, 3] + ΣΕΤ2

A[ΣΧ2, 2] <- A[ΣΧ2, 2] + ΣΕΤ2

A[ΣΧ2, 3] <- A[ΣΧ2, 3] + ΣΕΤ1

! Ενημέρωση βαθμολογίας (στήλη 1).

! Νικητής είναι αυτός που φτάνει τα 3 σετ.

ΑΝ ΣΕΤ1 = 3 ΤΟΤΕ

! Το ΣΧ1 κέρδισε (2 βαθμοί), το ΣΧ2 έχασε (1 βαθμός)

A[ΣΧ1, 1] <- A[ΣΧ1, 1] + 2

A[ΣΧ2, 1] <- A[ΣΧ2, 1] + 1

ΑΛΛΙΩΣ

! Το ΣΧ2 κέρδισε (2 βαθμοί), το ΣΧ1 έχασε (1 βαθμός)

A[ΣΧ1, 1] <- A[ΣΧ1, 1] + 1

A[ΣΧ2, 1] <- A[ΣΧ2, 1] + 2

ΤΕΛΟΣ_ΑΝ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

! Γ3: Ταξινόμηση σχολείων (Φθίνουσα)

! Κριτήριο 1: Βαθμολογία (A[... , 1])

! Κριτήριο 2: Σετ υπέρ (A[... , 2])

ΓΙΑ i ΑΠΟ 2 ΜΕΧΡΙ 5

ΓΙΑ j ΑΠΟ 5 ΜΕΧΡΙ i ΜΕ ΒΗΜΑ -1

! Έλεγχος συνθήκης για αντιμετάθεση (Bubble Sort)

**ΑΝ A[j-1, 1] < A[j, 1] Η (A[j-1, 1] =
A[j, 1] ΚΑΙ A[j-1, 2] < A[j, 2]) ΤΟΤΕ**

! Αντιμετάθεση στα Ονόματα

temp_ON <- ON[j-1]

ON[j-1] <- ON[j]

ON[j] <- temp_ON

! Αντιμετάθεση σε όλη τη γραμμή του πίνακα A

ΓΙΑ k ΑΠΟ 1 ΜΕΧΡΙ 3

temp_A <- A[j-1, k]

A[j-1, k] <- A[j, k]

A[j, k] <- temp_A

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΑΝ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

! Γ4: Εμφάνιση αποτελεσμάτων βάσει κατάταξης

ΓΙΑ *i* ΑΠΟ 1 ΜΕΧΡΙ 5

ΓΡΑΨΕ ΟΝ[*i*], Α[*i*, 1], Α[*i*, 2], Α[*i*, 3]

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΠΡΟΓΡΑΜΜΑΤΟΣ

ΘΕΜΑ Δ

ΠΡΟΓΡΑΜΜΑ θέμα_Δ_Σεμινάριο

! Δ4.α: Τμήμα δηλώσεων

ΜΕΤΑΒΛΗΤΕΣ

ΑΚΕΡΑΙΕΣ : ΑΠ[50, 6], ΑΠΤΡ[50, 2], *i*, θέση

ΧΑΡΑΚΤΗΡΕΣ : ΚΩΔ[50], αναζ_κωδ

ΑΡΧΗ

! Δ4.β: Κλήση διαδικασίας εισόδου δεδομένων

ΚΑΛΕΣΕ ΕΙΣ(ΚΩΔ, ΑΠ)

! Δ4.γ: Υπολογισμός απουσιών των δύο τριμήνων

ΓΙΑ *i* ΑΠΟ 1 ΜΕΧΡΙ 50

! Για το 1ο τρίμηνο ο αρχικός μήνας είναι ο 1ος

ΑΠΤΡ[*i*, 1] ← ΣΥΝΑΠ(*i*, ΑΠ, 1)

! Για το 2ο τρίμηνο ο αρχικός μήνας είναι ο 4ος

ΑΠΤΡ[*i*, 2] ← ΣΥΝΑΠ(*i*, ΑΠ, 4)

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

! Δ4.δ: Επαναληπτική αναζήτηση και έλεγχος δικαιώματος συμμετοχής

ΔΙΑΒΑΣΕ αναζ_κωδ

ΟΣΟ αναζ_κωδ <> "ΤΕΛΟΣ" **ΕΠΑΝΑΛΑΒΕ**

θέση <- ANAZ(αναζ_κωδ, ΚΩΔ)

ΑΝ θέση = 0 **ΤΟΤΕ**

ΓΡΑΨΕ "ΔΕΝ ΒΡΕΘΗΚΕ Ο ΚΩΔΙΚΟΣ"

ΑΛΛΙΩΣ

! Λιγότερες από 10 απουσίες σε ΚΑΘΕΝΑ από τα δύο τρίμηνα

ΑΝ ΑΠΤΡ[θέση, 1] < 10 **ΚΑΙ** ΑΠΤΡ[θέση, 2] < 10 **ΤΟΤΕ**

ΓΡΑΨΕ "Δικαιούται συμμετοχή στις εξετάσεις"

ΑΛΛΙΩΣ

ΓΡΑΨΕ "Δεν δικαιούται συμμετοχή στις εξετάσεις"

ΤΕΛΟΣ_ΑΝ

ΤΕΛΟΣ_ΑΝ

ΔΙΑΒΑΣΕ αναζ_κωδ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΠΡΟΓΡΑΜΜΑΤΟΣ

! -----

! Δ1. Διαδικασία ΕΙΣ

ΔΙΑΔΙΚΑΣΙΑ ΕΙΣ(ΚΩΔ, ΑΠ)

ΜΕΤΑΒΛΗΤΕΣ

ΑΚΕΡΑΙΕΣ : i, j, ΑΠ[50, 6]

ΧΑΡΑΚΤΗΡΕΣ : ΚΩΔ[50]

ΑΡΧΗ

ΓΙΑ i **ΑΠΟ** 1 **ΜΕΧΡΙ** 50

ΔΙΑΒΑΣΕ ΚΩΔ[i]

ΓΙΑ j **ΑΠΟ** 1 **ΜΕΧΡΙ** 6

ΔΙΑΒΑΣΕ ΑΠ[i, j]

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΤΕΛΟΣ_ΔΙΑΔΙΚΑΣΙΑΣ

```
! -----  
! Δ2. Συνάρτηση ANAZ (Σειριακή Αναζήτηση)  
ΣΥΝΑΡΤΗΣΗ ANAZ(Κλειδί, ΚΩΔ): ΑΚΕΡΑΙΑ  
ΜΕΤΑΒΛΗΤΕΣ  
    ΑΚΕΡΑΙΕΣ      : i, pos  
    ΧΑΡΑΚΤΗΡΕΣ    : Κλειδί, ΚΩΔ[50]  
    ΛΟΓΙΚΕΣ       : βρέθηκε  
ΑΡΧΗ  
pos <- 0  
βρέθηκε <- ΨΕΥΔΗΣ  
i <- 1  
ΟΣΟ i <= 50 ΚΑΙ βρέθηκε = ΨΕΥΔΗΣ ΕΠΑΝΑΛΑΒΕ  
    ΑΝ ΚΩΔ[i] = Κλειδί ΤΟΤΕ  
        βρέθηκε <- ΑΛΗΘΗΣ  
        pos <- i  
    ΑΛΛΙΩΣ  
        i <- i + 1  
ΤΕΛΟΣ_ΑΝ  
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ  
  
ANAZ <- pos  
ΤΕΛΟΣ_ΣΥΝΑΡΤΗΣΗΣ
```

```
! -----
! Δ3. Συνάρτηση ΣΥΝΑΠ
ΣΥΝΑΡΤΗΣΗ ΣΥΝΑΠ(γραμμή, ΑΠ, αρχ_μήνας): ΑΚΕΡΑΙΑ
ΜΕΤΑΒΛΗΤΕΣ
    ΑΚΕΡΑΙΕΣ : γραμμή, ΑΠ[50, 6], αρχ_μήνας, i, αθροισμα_απουσιών
ΑΡΧΗ
αθροισμα_απουσιών <- 0
! Υπολογίζει το άθροισμα από τον αρχικό μήνα έως και τον μεθεπόμενο
! (άρα 3 μήνες)
ΓΙΑ i ΑΠΟ αρχ_μήνας ΜΕΧΡΙ αρχ_μήνας + 2
    αθροισμα_απουσιών <- αθροισμα_απουσιών + ΑΠ[γραμμή, i]
ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ

ΣΥΝΑΠ <- αθροισμα_απουσιών
ΤΕΛΟΣ_ΣΥΝΑΡΤΗΣΗΣ
```

ΚΑΛΗ ΕΠΙΤΥΧΙΑ!

George W. Aravidis

About : <https://medium.com/@georgearavidis/about>

Github : <https://github.com/inbitwetrust/aepp>

SpeakerDeck : <https://speakerdeck.com/georgearavidis>

Το παρόν αρχείο αποτελεί αποτέλεσμα προσωπικής επιμέλειας, οργάνωσης και μορφοποίησης του εκπαιδευτικού υλικού που αφορά τα θέματα του μαθήματος ΑΕΠΠ για το έτος 2023. Η δημιουργία του απαιτεί σημαντικό χρόνο και προσπάθεια, με σκοπό να προσφερθεί ένα σαφές και αξιόπιστο βοήθημα σε μαθητές και εκπαιδευτικούς.

Το υλικό διατίθεται ελεύθερα για εκπαιδευτική χρήση και μπορεί να αναπαραχθεί, να αποθηκευτεί ή να κοινοποιηθεί σε ιστοσελίδες, ψηφιακές πλατφόρμες και άλλους χώρους διαμοιρασμού. Η διανομή του αρχείου επιτρέπεται υπό την προϋπόθεση ότι το αρχείο θα παραμένει απολύτως αυτούσιο και δεν θα έχει υποστεί καμία απολύτως τροποποίηση.

Απαγορεύεται ρητά οποιαδήποτε αλλαγή στη δομή, στο περιεχόμενο, στη μορφοποίηση, στη διάταξη ή στη γενικότερη παρουσίαση του αρχείου. Δεν επιτρέπεται η επεξεργασία του αρχείου, η μετατροπή του σε άλλη μορφή με αλλοιωμένο περιεχόμενο, ούτε η ενσωμάτωσή του σε άλλο υλικό με τρόπο που μεταβάλλει την αρχική του μορφή.

Ιδιαίτερα επισημαίνεται ότι δεν επιτρέπεται η αφαίρεση, απόκρυψη, αντικατάσταση ή αλλοίωση των στοιχείων του δημιουργού, των αναφορών στην ιστοσελίδα προέλευσης ή οποιαδήποτε πληροφορίας ταυτοποίησης που περιλαμβάνεται στο παρόν αρχείο. Η δημοσίευση του υλικού χωρίς τα στοιχεία αυτά ή με τροποποιημένη παρουσίαση θεωρείται παραβίαση των όρων χρήσης.

Για τη συγγραφή και μορφοποίηση των προγραμμάτων που περιλαμβάνονται στο παρόν αρχείο χρησιμοποιήθηκε το λογισμικό GloEditor www.gloeditor.gr, το οποίο αποτελεί ένα χρήσιμο εργαλείο για την ανάπτυξη και παρουσίαση προγραμμάτων στη γλώσσα προγραμματισμού ΓΛΩΣΣΑ.

Το παρόν υλικό διατίθεται με άδεια χρήσης Creative Commons Αναφορά Δημιουργού – Μη Εμπορική Χρήση – Όχι Παράγωγα Έργα (CC BY-NC-ND). Η χρήση, αναπαραγωγή ή διανομή του αρχείου συνεπάγεται την πλήρη αποδοχή των παραπάνω όρων και την υποχρέωση τήρησής τους.